PGAUTHZ · AUTHORIZATION INSIDE POSTGRESQL

Can your AuthZ do that?



A tour of real authorization use cases, from classic RBAC to AI agents, and how pgauthz solves each one.

Thomas Darimont · github.com/thomasdarimont/pgauthz

THE QUESTION

Every access decision has four parts

P

Principal

A user, a service, or an AI agent

A

Action

read, edit, transfer, or a tool call

R

Resource

A document, an account, a customer

C

Conditions

Everything else: roles, attributes, relationships, history

The access models differ in **where the conditions come from**. That is the thread through this talk.

THE ROUTE

Six questions, one engine

RBAC
Which role do you hold?

ABAC
Which attributes hold right now?

PBAC
What does the central policy say?

ReBAC
How are you related to this resource?

Temporal
What have you already done?

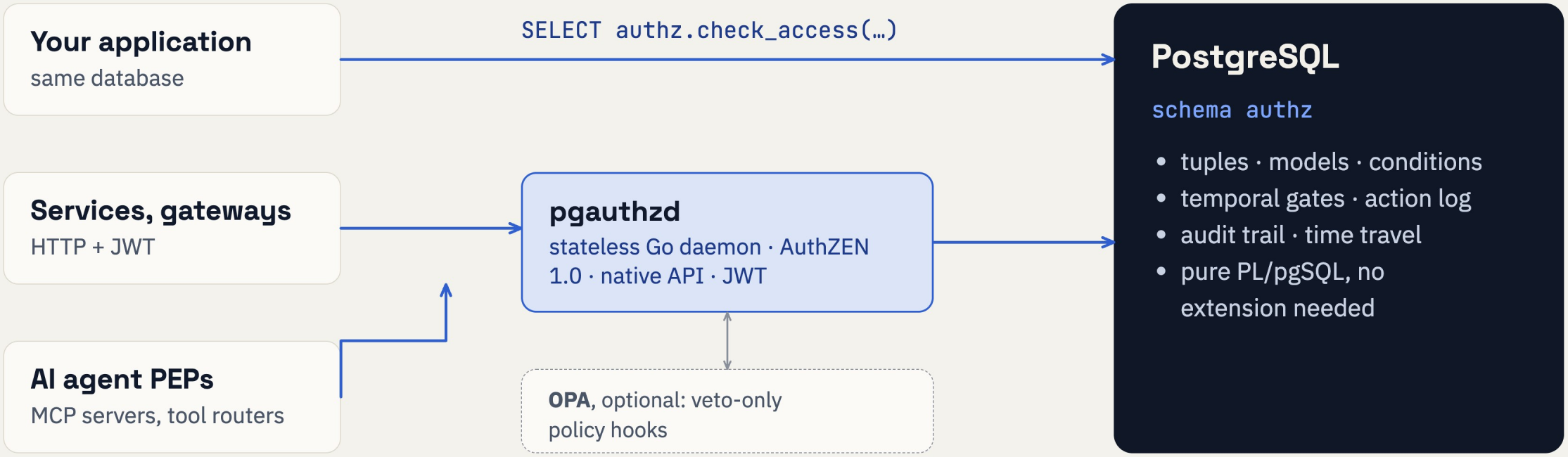
Agentic
What may this agent do, for this task?

Real systems need **all of them at once**, not a different product for each.

MEET PGAUTHZ

An authorization engine inside PostgreSQL

The Zanzibar / OpenFGA model as SQL functions, plus a small daemon that puts it on the network.



BUILDING BLOCKS

Tuples are the data, rules are the model

`user:alice` — `editor` → `document:readme` one row = one relationship

Direct

Granted by writing a tuple

`editor: [user]`

Computed

Implied by another relation on the same object

`can_read: viewer or editor`

Tuple-to-userset

Inherited over a link to another object

`viewer from parent`

Plus **AND / BUT NOT** groups, **wildcards**, **conditions** and **gates**. OpenFGA models import as they are.

USE CASE 01 · RBAC

Editors write, viewers read

> Editors may change a document.
> Viewers may only read it.

- A role is a relation on the resource type
- Assigning a role writes one tuple
- Permissions are computed from roles
- Revoking is one delete, audited

`direct` `computed` `check_access`

OPENFGA DSL + SQL

```
type document
  relations
    define viewer: [user]
    define editor: [user]
    define can_read: viewer or editor
    define can_write: editor

-- role assignment = one tuple
SELECT authz.write_tuple('helloworld',
  'user','alice','editor','document','readme');

SELECT authz.check_access('helloworld',
  'user','bob','can_write','document','readme');
-- ⇒ false (viewers don't write)
```

USE CASE 02 · RBAC AT SCALE

Role hierarchies without role explosion

> GitHub-style repo roles, whole teams as grantees, and an auditor who sees every document.

- Hierarchy = a chain of computed relations
- Groups are usersets: team#member
- Org roles flow down to every repo
- Global roles: one object-wildcard tuple

usersets · object wildcard · OpenFGA import

SIMPLIFIED FROM EXAMPLES/MODELS/GITHUB

```
type repo
  relations
    define owner: [organization]
    define admin: [user, team#member]
        or repo_admin from owner
    define writer: [user, team#member] or admin
    define reader: [user, team#member] or writer
write_tuple('team','eng','writer','repo','api',
  p_user_relation ⇒ 'member')
-- nadia audits every document, now and future
write_tuple('user','nadia','viewer','document','*')
```

```
check(user:dave, reader, repo:api)
→ allow · dave ∈ team:eng → writer → reader
```

USE CASE 03 · ABAC

Only from the office network

> Bob may read the payroll report, but only from 10.0.0.0/8 and during office hours.

- A condition is a named SQL or CEL expression
- Bound per tuple, evaluated at check time
- Request context meets stored context
- validate_condition dry-runs it first

conditions · SQL · CEL · sandboxed

```
SQL · $1 = REQUEST, $2 = STORED

SELECT authz.create_condition_sql('demo',
  'ip_in_range', $$($1⟶»'client_ip')::inet
    «⟸ ($2⟶»'allowed_cidr')::cidr$$);

SELECT authz.write_tuple('demo', 'user','bob',
  'viewer','document','payroll',
  p_condition ⇒ 'ip_in_range',
  p_condition_context ⇒
    '{"allowed_cidr": "10.0.0.0/8"}');

SELECT authz.check_access_with_context('demo',
  'user','bob','viewer','document','payroll',
  '{"client_ip": "10.1.2.3"}');  -- ⇒ true
```

USE CASE 04 · TIME-BOXED ACCESS

A contractor for 30 days, then gone

```
write_tuple('demo','user','carol','viewer','doc','plan',
  p_expires_at ⇒ now() + interval '30 days')
```

Tool	Use it when	What you get
expires_at on the tuple	Plain "grant until T"	Server time decides; enforced on checks and searches
Condition	Business hours, recurring windows	Flexible, but the caller supplies the time
Scheduled delete	Revocation must reach the changefeed	Explicit, audited event, as timely as your job
Contextual tuple	Grant lives for one request	Nothing stored at all

USE CASE 05 · ABAC, FAIL-CLOSED

No access from unmanaged devices

> Employees view documents only from a managed device. Device info missing? Deny, don't guess.

- One conditional wildcard tuple covers every employee and document
- Intersected into each employee path
- Missing context never grants: the PEP learns what it forgot

`intersection` `check_access_detailed`

FROM EXAMPLES/MODELS/AIA-ACME

```
type document
  relations
    define managed_device: [employee:*]
    define view: customer_readers
      or (employee_can_view and managed_device)

-- the global constraint is ONE tuple
write_tuple('aia_acme','employee','*',
  'managed_device','document','*',
  p_condition ⇒ 'managed_device')
```

```
check(employee:bob, view, document:q3-plan, {})
→ conditional · missing_context: request.device
```

USE CASE 06 · PBAC

Exports need the ”exporter” role

> Security wants org-wide rules as code: exports need a Keycloak client role, finance docs only 08:00 to 18:00.

- Hooks are Rego files, global or per store
- Veto-only: they narrow, never widen
- Read verified claims and server time
- deny_write governs writes, too

OPA hooks · Rego · JWT claims

REGO · EXAMPLES/OPA-HOOKS/GLOBAL

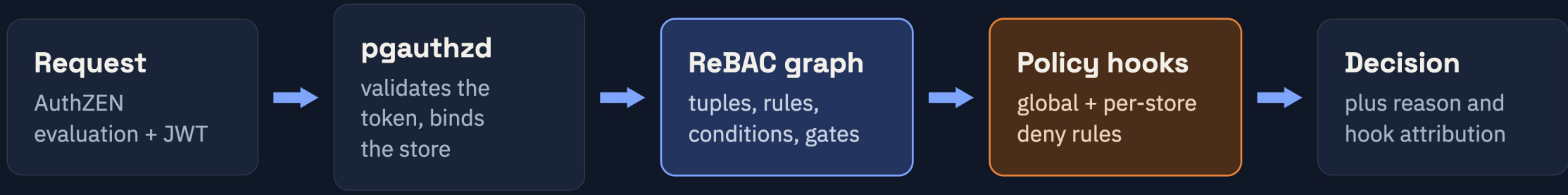
```rego
package authz.hooks.v1.global.claims_guard
import data.authz.hooks.lib.v1.keycloak

deny contains {"code": "export_requires_client_role"} if {
  input.action == "can_export"
  not keycloak.has_client_role(input.actor,
      "document-api", "exporter")
}
# business_hours.rego reads input.evaluated_at:
# server time, never the caller's clock
```

```
check(alice, can_export, document:doc_1)
→ deny · 403, global hook claims_guard
```

PBAC + REBAC

The graph grants, policies veto




```
allow = graph allows AND no hook denies
```

Domain teams own models and tuples in their store.

Security owns the Rego, validated before mount.

Search stays honest: actions filtered, enumeration refused by default.

USE CASE 07 · POLICIES AS DATA

”Our policies are already in Cedar”

Cedar (Authorization in Action, ch. 9)	pgauthz
Entity attribute: owner, manager	A relation, written as a tuple
principal in resource.readers_team	Userset: team#employee_member
resource.owner.manager == principal	Tuple-to-userset: manager from owner
forbid when device not managed	Required relation (AND) with a condition
classification == "Legal"	Reified object: one tuple on classification:legal
Policy template	A tuple

Bonus: **list_objects** and **list_subjects** answer "which documents?" and "who?" over the same rules. No partial evaluation needed.

USE CASE 08 · REBAC

Share a folder, everything inside follows

> Google-Drive-style sharing on nested folders, with public links.

- "from parent" walks the tree recursively
- Stable ids: rename costs 0 writes, move costs 1
- Public link: the user:* wildcard
- 15-level chain: about 1.5 ms

`tuple-to-userset` `user:*`

OPENFGA DSL

```
type folder
  relations
    define parent: [folder]
    define viewer: [user, group#member]
    define can_view: viewer or can_view from parent
type doc
  relations
    define parent: [folder]
    define viewer: [user, user:*, group#member]
    define can_read: viewer or can_view from parent
```

```
check(user:ann, can_read, doc:plan)
→ allow · viewer on folder:root, 3 levels up
```

USE CASE 09 · REBAC RULE GROUPS

Delete your own items, unless you’re admin

> Delete items you own on lists you manage. Admins delete anything. Blocked members can't comment.

- Rule groups: OR, AND, BUT NOT
- Groups are OR'd with each other
- An unevaluable ban still bans

`intersection` `exclusion` `AuthZEN interop`

OPENFGA DSL

```
# AuthZEN interop "todo" model
type todo
  relations
    define can_delete_todo:
      (can_manage_todo_items from parent
       and owner)
      or admin from parent
# members may comment, unless blocked
type resource
  relations
    define can_comment: member but not blocked
```

```
check(user:sam, can_delete_todo, todo:t7)
→ deny · intersection_unsatisfied
```

USE CASE 10 · DATA FILTERING

Show me only what I may see

> List the 20 newest documents Bob can read, paged, in one query.

- list_objects, list_subjects, list_actions
- Co-located: authorization is a JOIN
- Over HTTP: AuthZEN search, filter by ids
- Cost tracks Bob's reach, not table size

`search API` `is_wildcard`

SQL · EXAMPLES/FILTERING

```sql
WITH authorized AS MATERIALIZED (
  SELECT object_id, is_wildcard
    FROM authz.list_objects('demo',
      'internal_user','bob','can_read','document')
)
SELECT d.*
  FROM documents d     -- your own table
 WHERE EXISTS (SELECT 1 FROM authorized
               WHERE is_wildcard)
    OR d.id IN (SELECT object_id FROM authorized)
 ORDER BY d.created_at DESC
 LIMIT 20;
```

USE CASE 11 · AUDIT AND FORENSICS

Could Grace read payroll back then? Why?

> Grace left the payroll team. Could she read payroll before that, and who removed her?

- Today: deny, and explain says why
- At T: the answer replayed from the audit log
- The audit row shows who changed what, when
- explain is live: it traces today, not T

explain · time travel · audit trail

SQL · DEMO STORE

```
SELECT authz.check_access('demo','internal_user',
  'grace','can_read','document','doc_payroll_001');
-- ⇒ false (today)
-- why not? live explain, as of today
SELECT authz.explain_access(…)⟶»'summary';
-- … = DENIED (no_matching_rule)
--   ✗ [no_direct_tuple] viewer: no tuple
--   ✗ [computed] ← internal_collaborator: no tuple
SELECT authz.audit_check_access('demo', …,
  '<T: before the revoke>');   -- ⇒ true
SELECT * FROM authz.audit_list_user('demo',
  'internal_user','grace');
-- DELETE member team:payroll_team,
--   performed_by=…, performed_at=…
```

TEMPORAL GATES

Beyond point in time: what did you already do?

`formerly_within`

It happened in the window: prior approval, MFA freshness

`count_within`

At most N: rate limits, lockouts, "never" with max 0

`count_distinct_within`

At most N different files, accounts or partners

`sum_within`

A total stays under a cap: spend limits

PEP runs the action → **record_event** → next check: graph allows → **gates may veto**

Gates never grant. They apply to every check, listing, explain and time-travel path.

USE CASE 12 · TEMPORAL GATES

5 transfers/h, €5,000, approved first

> A velocity backstop on transfers, plus a recent approval by the approvals service.

- Any failing clause denies: gate_denied
- Payload schemas: recorders must send amount
- mode: shadow logs would-be denials first
- reserve_event: exact caps under concurrency

`add_gate` `record_event`

SQL · MODEL_DESIGN §17

```
SELECT authz.add_gate('bank','account','transfer',
 'velocity_backstop', '{"all_of": [
  {"count_within": {"window": "1h", "kind": "request",
     "max": 5, "plus": 1}},
  {"sum_within": {"window": "1h", "kind": "response",
     "field": "input.amount", "max": 5000,
     "plus": "$request.input.amount"}},
  {"formerly_within": {"window": "1h",
     "action": "approve_sale", "kind": "response",
     "match": {"output.approved": true},
     "recorded_by": ["svc:approvals"]}}]}');

check(alice, transfer, account:acc-1, {amount: 1200})
→ deny · gate_denied: 4,300 + 1,200 > 5,000
```

USE CASE 13 · ONE MECHANISM, MANY RULES

Four-eyes, step-up, lockout, quotas

Separation of duties

On doc#approve: whoever submitted this doc may not approve it

`count_within action: submit, scope: object, max: 0`

Step-up freshness

Sensitive action only with an MFA event in the last 15 minutes

`formerly_within window: 15m, action: mfa`

Lockout

Too many refused attempts lock the subject out for a while

`count_within kind: denied, max: N`

Download quotas

A daily cap per user, plus a limit per file (gdrive example)

`calendar: day, tz · scope: object`

AGENTIC AUTHORIZATION

Every tool call is an access check

Agent plans a tool call → PEP builds the PARC request → pgauthzd decides, with a reason → Execute, or replan

`intersection_unsatisfied`
Outside the task's scope

`gate_denied`
A history rule held: not yet, or too often

`conditional`
missing_context names what to supply

Plan before acting: **search/action** returns the permitted tool set, computed by the same rules. Guidance equals enforcement.

USE CASE 14 · AGENT TASK SCOPE

Only the customer this task is about

> ACME Assist may work for CustCo and Globex, but this task is about CustCo only.

- The task is a relationship: task_scope
- Stored with expires_at for the task's life
- Or a contextual tuple, for one request
- Globex docs: intersection_unsatisfied

`expiring tuples` `contextual tuples`

EXAMPLES/MODELS/AGENTS

```
type customer
  relations
    define agent_reader: [agent#reach]
    define task_scope:   [agent]
    define agent_scope:  agent_reader and task_scope
type customer_doc
  relations
    define customer: [customer]
    define documents_read: member from customer
      or agent_scope from customer
agent:acme_assist  task_scope  customer:CustCo  8h
```

```
check(acme_assist, documents_read, globex-plan)
→ deny · intersection_unsatisfied
```

USE CASE 15 · AGENT GUARDRAILS

Email only after a summary exists

> Send email only within an hour of a summary. At most 20 tool calls per hour.

- Task state lives in the action log, not the prompt
- The recorder is the PEP, never the agent
- send_email stays hidden until the gate passes

`formerly_within` `count_within`

HTTP + SQL

```
# PEP, after the summarize tool completed
POST /stores/agents/pgauthz/v1/events
{"subject_id": "acme_assist",
 "action": "documents_summarize", "kind": "response"}
-- email only within an hour of a summary
SELECT authz.add_gate('agents','mailbox',
  'send_email','summary_first',
  '{"all_of": [{"formerly_within": {
    "window": "1h",
    "action": "documents_summarize",
    "kind": "response"}}]}');
```

```
check(acme_assist, send_email, mailbox:acme)
→ deny · gate_denied until a summary is recorded
```

USE CASE 16 · DELEGATION

Subagents never exceed their parent

> ACME Assist hands research to a subagent for 4 hours. Revoke the parent, the child loses access too.

- Grants go to an agent's reach userset
- Delegates read through the parent's grant
- Over-broad delegation is refused on write

`write-checked` `attenuation`

DSL + HTTP (ABRIDGED)

```
type agent
  relations
    define delegate: [agent]
    define reach: [agent] or reach from delegate
POST /stores/agents/pgauthz/v1/write-checked
{"preconditions": [{"match": "allowed",
   "user_id": "acme_assist", "relation": "agent_scope",
   "object_type": "customer", "object_id": "CustCo"}],
 "writes": [{"user_id": "research_agent",
   "relation": "delegate", "object_id": "acme_assist",
   "expires_at": "2026-10-01T18:00:00Z"}, …]}
```

```
check(research_agent, documents_read, custco-briefing)
→ allow · deny once acme_assist loses its grant
```

USE CASE 17 · AGENTS, CONTINUED

Guard the control plane, filter retrieval

Protect by absence

Agents may run automations, never create or change them.

```
type automation
  relations
    define run:    [agent, user]
    define create: [user]
    define update: [user]
```

No agent path exists, and writing one is rejected. Agent PEPs talk to a decision-only pgauthzd.

Authorize before retrieval

Once a chunk is in the prompt, it can leak.

Filter, then search: list_objects ids become the vector store's metadata filter.

Search, then check: top-k ids only, one check-batch, then fetch content.

Same documents_read relation, so task scope applies for free.

THE KILLER USE CASE · ONE SENTENCE, FOUR QUESTIONS

”Share this folder with the marketing team, except Bob; give external reviewers read-only until next Friday, but only once they have accepted the NDA; cap downloads at three a day; and any payment over 10k needs two distinct approvers.”

Who are you?

RBAC

the marketing team: team:marketing#member

How are you related?

ReBAC

folder → doc, BUT NOT blocked for Bob

What is true right now?

ABAC

expires_at on the reviewer tuple

What already happened?

`temporal gates`

NDA first, three a day, two approvers

One store, three types, three gates

THE GRAPH

```
type folder
  relations
    define editor:   [user, team#member]
    define reviewer: [user]
    define blocked:  [user]
    define can_read: editor but not blocked
    define review:   reviewer but not blocked
type doc
  relations
    define parent:   [folder]
    define can_read: can_read from parent
    define review:   review from parent
    define download: can_read or review
```

PAYMENTS + GATES

```
type payment
  relations
    define requester:     [user]
    define execute:       requester
    define execute_large: requester
# doc#review · nda_first
#  formerly_within accept_nda, 30 days
# doc#download · three_a_day
#  count_within calendar day, max 3
# payment#execute_large · four_eyes
#  distinct approvers ≥ 2
#  AND none of them is $request.self
```

Tuples for the graph, events for history

SEED.SQL

```
team:marketing#member  editor  folder:campaign
user:bob    blocked   folder:campaign
user:carol  reviewer  folder:campaign  +7 days
user:erin   reviewer  folder:campaign  +7 days
folder:campaign  parent  doc:brief
user:dave   requester payment:p1  12,000

# action log, written by trusted services
carol  accept_nda          svc:review-portal
dave   approval_received   approver: carol
dave   approval_received   approver: grace
```

FOUR_EYES (ABRIDGED)

```
SELECT authz.add_gate('fourq','payment',
 'execute_large','four_eyes','{"all_of": [
 {"count_distinct_within": {
   "action": "approval_received",
   "scope": "object", "window": "30 days",
   "key": "payload.input.approver",
   "min": 2}},
 {"count_within": {
   "action": "approval_received",
   "scope": "object", "window": "30 days",
   "match": {"input.approver":
             "$request.self"},
   "max": 0}}]}');
```

Now ask it

Check	Answer	Why
alice can_edit doc:brief	allow	team:marketing#member → editor
bob can_read doc:brief	deny	excluded: blocked
erin review doc:brief	deny	gate_denied: nda_first
… after the portal records her NDA	allow	reviewer, NDA on record
carol review, after next Friday	deny	reviewer tuple expired
alice's 4th download today	deny	gate_denied: three_a_day
dave execute_large p1, no approvals	deny	gate_denied: four_eyes
… after carol and grace approve	allow	two distinct approvers, neither dave
… without self in the context	deny	gate_missing_context: fails closed

Listings agree: who can read the brief? alice, dave. A capped alice lists nothing.

RECAP

Six models, one engine, one database

RBAC

Roles as relations, computed hierarchies, usersets, object wildcards

ABAC

SQL or CEL conditions, expires_at, contextual tuples, fail-closed

PBAC

Veto-only OPA hooks on verified claims; policies as data

ReBAC

Tuple-to-userset, AND / BUT NOT, search as a JOIN, time travel

Temporal

Action log, four gate primitives, shadow mode, reserve_event

Agentic

Task scope, sequencing, delegation with attenuation, RAG filtering

Every one of them: **explainable**, **audited**, and **replayable at any point in time**.

OPERATING IT

Built for platform teams

Just PostgreSQL

Pure PL/pgSQL. Your backups, replicas and transactions.

pgauthzd

AuthZEN 1.0, native API, multiple JWT issuers, metrics, decision log

Multi-tenant

Stores per tenant; namespace write control enforced in the database

Model as code

pgauthzctl: DSL in git, YAML tests in CI, registry canary and rollout

Fast enough

Sub-millisecond checks; 10-deep nested teams in about 0.5 ms

See it

Playground UI, Helm chart, read replicas with freshness tokens

Yes. And it's just PostgreSQL.

Try it: **./bootstrap.sh** starts the stack, loads the examples and runs the tests.

`github.com/thomasdarimont/pgauthz`

Questions?